

SettleIQ — System Architecture

1. Architecture Goal

SettleIQ will use a **modular monolithic architecture**.

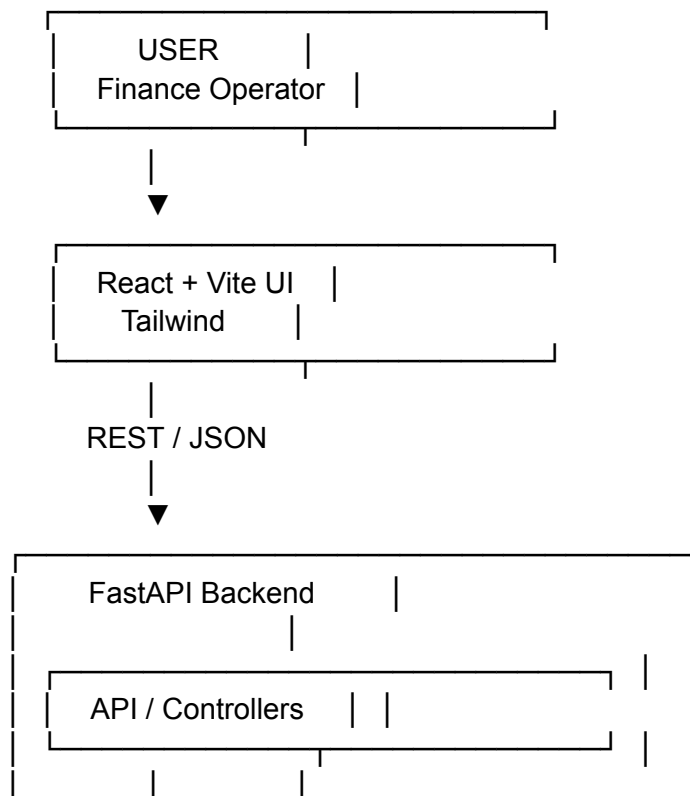
The system will have:

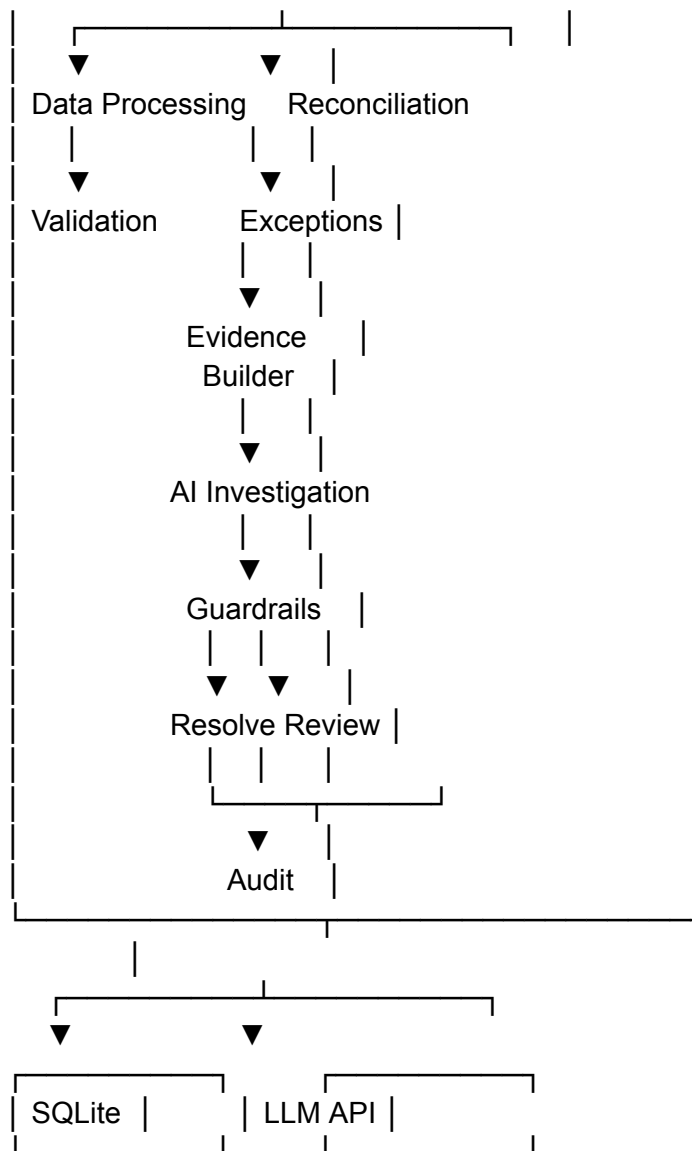
- One React frontend
- One FastAPI backend
- One SQLite database
- One external LLM service

The backend will remain internally modular so that each responsibility can be developed, tested, and modified independently.

The architecture is intentionally simple because the goal is to build a reliable, working system within the Buildathon timeline rather than introduce unnecessary enterprise complexity.

2. High-Level Architecture





3. Frontend

Technology

React + Vite + Tailwind CSS

The frontend is responsible for:

- Displaying information
- Accepting user actions
- Calling backend APIs
- Presenting reconciliation results
- Displaying AI investigation results
- Providing the human review interface

The frontend should **not contain financial reconciliation logic**.

All important financial processing and decision-making will happen in the backend.

Main Frontend Pages

3.1 Dashboard

Displays:

- Total records processed
- Matched records
- Exceptions
- Auto-resolved cases
- Human-review cases
- Match rate
- Auto-resolution rate

3.2 Data Upload

Allows the finance operator to upload:

payments.csv
settlements.csv
fees.csv

The user can then:

Validate → Start Reconciliation

3.3 Reconciliation Results

Displays transaction-level results.

Example:

Transaction	Payment	Settlement	Status	Exception	
PAY_10001	₹5,000	₹5,000	MATCHED	—	
PAY_10002	₹5,000	₹4,850	RESOLVED	Amount / Fee	

PAY_10003	₹3,000	—	REVIEW	Missing Settlement	
-----------	--------	---	--------	--------------------	--

3.4 Exception Investigation

Displays:

- Transaction information
 - Exception type
 - Relevant financial records
 - Evidence
 - AI investigation
 - Root cause
 - Confidence
 - Recommended action
 - Final resolution
-

3.5 Review Queue

Displays cases that require human intervention.

The reviewer can:

- Approve
 - Reject
 - Keep unresolved
-

3.6 Settlement Q&A

This is an **advanced feature**.

The finance operator can ask questions such as:

"Why is today's settlement ₹68,600 lower than expected?"

The system retrieves relevant financial data and generates an evidence-backed answer.

4. Backend

Technology

Python + FastAPI

The backend contains all core business logic.

The backend will be organized into separate modules:

```
backend/
├── app/
│   ├── api/
│   ├── models/
│   ├── schemas/
│   ├── services/
│   ├── reconciliation/
│   ├── ai/
│   ├── guardrails/
│   ├── database/
│   └── utils/
```

5. API Layer

The API layer receives requests from the React frontend.

Initial API endpoints:

POST /api/upload

POST /api/reconcile

GET /api/dashboard

GET /api/transactions

GET /api/exceptions

GET /api/exceptions/{id}

POST /api/exceptions/{id}/investigate

POST /api/exceptions/{id}/review

GET /api/audit

POST /api/ask

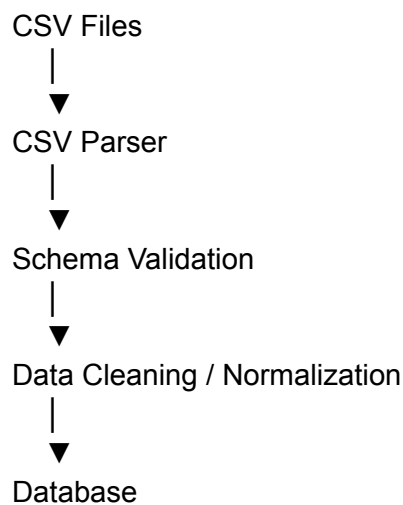
We will avoid creating unnecessary endpoints and expand the API only when required by the product.

6. Data Ingestion Module

The Data Ingestion module is responsible for:

- Reading CSV files
- Validating required columns
- Validating data types
- Detecting malformed records
- Detecting duplicates
- Cleaning/normalizing valid data
- Storing valid records

Flow



7. Reconciliation Engine

The Reconciliation Engine is one of the most important components.

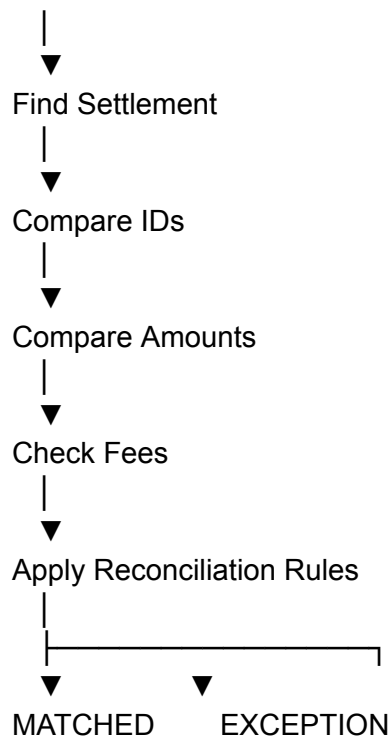
It will initially be **fully deterministic**.

It will:

1. Identify the corresponding settlement for a payment.
2. Compare identifiers.
3. Compare amounts.
4. Check applicable fees.
5. Apply reconciliation rules.
6. Classify the result.

Flow

Payment



The Reconciliation Engine **does not call the LLM**.

This makes the financial matching process:

- Predictable
- Testable
- Fast
- Inexpensive
- Easier to debug

8. Exception Engine

When the reconciliation engine cannot successfully match a transaction, the Exception Engine classifies the issue.

Supported exception types

AMOUNT_MISMATCH
MISSING_SETTLEMENT
DUPLICATE
REFERENCE_MISMATCH
UNKNOWN

Each exception receives a unique identifier.

Example:

EXC_000142

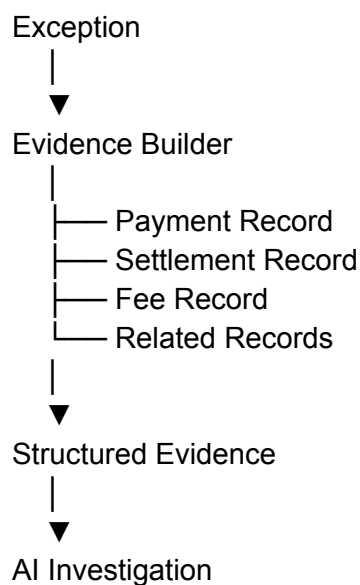
The exception will also reference the related transaction and relevant financial records.

9. Evidence Builder

The Evidence Builder connects the deterministic reconciliation system with the AI investigation layer.

Instead of giving the LLM access to the entire database, the backend retrieves only the records relevant to the specific exception.

Flow



This makes the AI system more controlled and predictable.

10. AI Investigation Service

The AI Investigation Service is responsible for investigating exceptions after deterministic reconciliation has identified them.

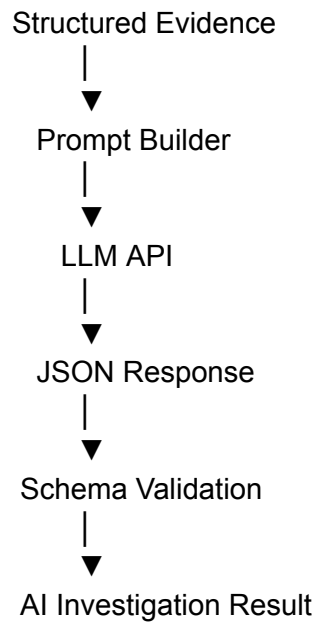
AI responsibilities

The AI can:

- Investigate the exception

- Gather meaning from the provided evidence
- Identify the likely root cause
- Generate an explanation
- Assign a confidence score
- Recommend an action
- Identify the evidence supporting its conclusion

AI flow



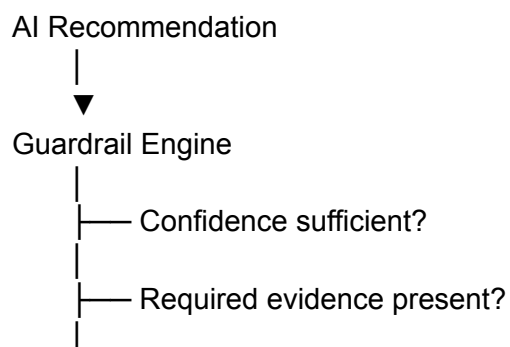
The AI should return structured output rather than unrestricted text.

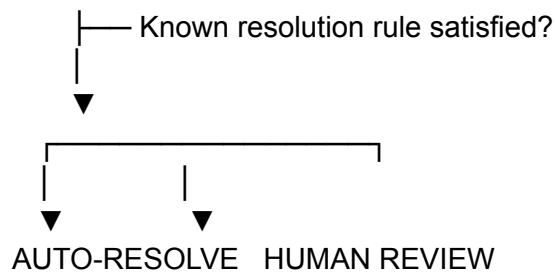
11. Guardrail Engine

The Guardrail Engine is a critical component of SettleIQ.

The AI **does not directly modify financial records or directly resolve exceptions.**

Instead:





For example:

AI Recommendation:
AUTO_RESOLVE

Confidence:
0.96



Required evidence present ✓
Confidence threshold passed ✓
Known resolution rule satisfied ✓



AUTO-RESOLVE

If any important condition fails:

→ HUMAN REVIEW

The AI therefore operates under **bounded autonomy**.

12. Review Service

The Review Service handles cases that cannot safely be resolved automatically.

Review workflow

Exception



AI Recommendation



Human Review



- Approve
- Reject
- Keep Unresolved

Every human action is recorded in the database and audit trail.

13. Audit Service

The Audit Service records important system and user actions.

Example audit events:

EXCEPTION_CREATED
AI_INVESTIGATION_COMPLETED
AUTO_RESOLVED
SENT_TO_REVIEW
HUMAN_APPROVED
HUMAN_REJECTED

A typical audit record should contain:

Transaction ID
Exception ID
Action
Evidence Used
AI Recommendation
Confidence
Final Decision
Actor
Timestamp

This provides traceability for every important decision.

14. Database

MVP Database: SQLite

SQLite is appropriate for the MVP because:

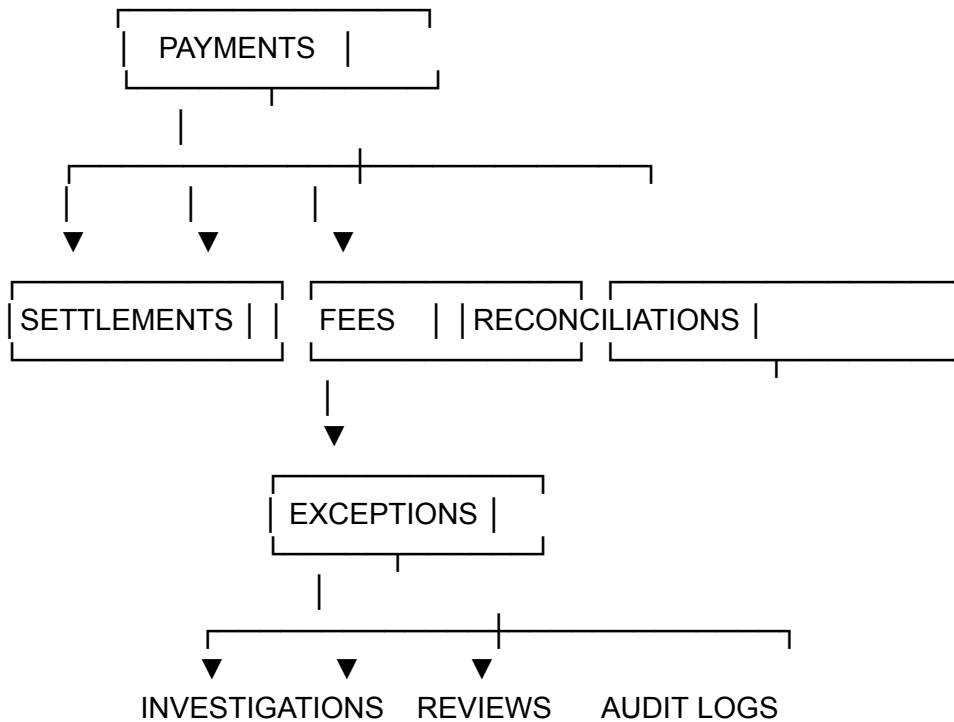
- It requires no separate database server.
- It is easy to set up.
- The dataset is synthetic and relatively small.

- It is sufficient for development and demonstration.
- It is easy to understand and deploy.

Main tables

payments
 settlements
 fees
 reconciliations
 exceptions
 investigations
 reviews
 audit_logs

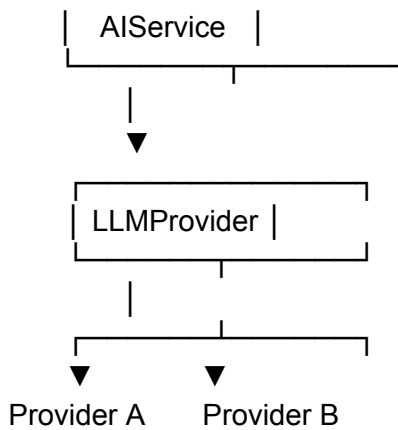
15. Database Relationships



16. AI Provider Architecture

The application should remain **LLM-provider agnostic**.

Instead of directly calling a specific provider throughout the application, we will create an internal AI service interface.



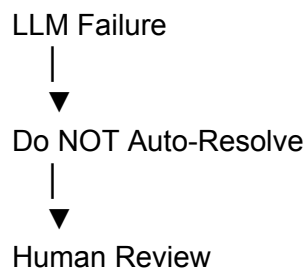
Initially, we will implement only one provider.

This allows us to change the LLM later without rewriting the entire application.

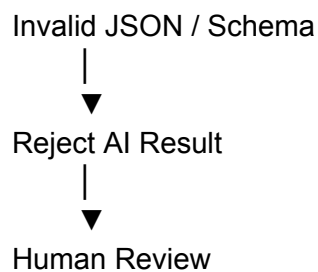
17. Failure Handling

Failure handling is part of the architecture rather than something added at the end.

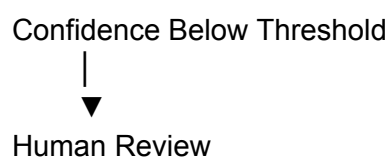
Scenario 1 — LLM unavailable



Scenario 2 — Invalid AI response



Scenario 3 — Low confidence



Scenario 4 — Missing evidence

Insufficient Evidence



Human Review

Scenario 5 — Database/API failure

The system should fail safely rather than partially modifying financial state.

18. Settlement Q&A Architecture — Advanced Feature

Settlement Q&A will be added only after the core MVP is working.

Flow

User Question



Question Understanding



Relevant Data Retrieval



Deterministic Financial Calculations



Evidence Package



LLM



Answer + Evidence

Critical financial calculations should be performed by the backend rather than relying on the LLM.

For example:

Expected Settlement - Actual Settlement

=

Shortfall

The backend calculates the amount.

The LLM explains and contextualizes the result.

Principle:

Code calculates. AI investigates, reasons, and explains.

19. End-to-End Example

Consider the following transaction:

Payment: ₹5,000
Settlement: ₹4,850
Fee: ₹150

Step 1 — Upload

The user uploads:

payments.csv
settlements.csv
fees.csv

Step 2 — Validation

FastAPI validates the uploaded data.

Step 3 — Reconciliation

The Reconciliation Engine detects the amount discrepancy.

Step 4 — Exception Classification

The Exception Engine creates:

Exception Type:
AMOUNT_MISMATCH

Step 5 — Evidence Collection

The Evidence Builder retrieves:

Payment Record
Settlement Record
Fee Record

Step 6 — AI Investigation

The AI returns:

Root Cause:
PROCESSING_FEE

Confidence:
0.96

Recommendation:
AUTO_RESOLVE

Step 7 — Guardrail

The Guardrail Engine checks:

Confidence sufficient ✓
Required evidence exists ✓
Fee explains difference ✓
Known resolution rule satisfied ✓

Step 8 — Resolution

AUTO-RESOLVED

Step 9 — Audit

The decision is recorded.

Step 10 — Dashboard

The reconciliation metrics are updated.

20. Technology Stack — Final

Layer	Technology
Frontend	React
Build Tool	Vite

Styling	Tailwind CSS
Backend	Python
API Framework	FastAPI
Data Processing	Pandas
Validation	Pydantic
Database	SQLite
ORM	SQLAlchemy
AI	LLM API
Communication	REST / JSON
Version Control	Git + GitHub
Development Environment	VS Code
AI Coding Assistance	GitHub Copilot / Free AI Tools
Deployment	To be decided after MVP

The technology stack is intentionally straightforward. The project should demonstrate **engineering quality and AI judgment**, not unnecessary infrastructure complexity.

21. Repository Structure

```

settleiq/
├── frontend/
├── backend/
│   ├── app/
│   │   ├── api/
│   │   ├── models/
│   │   ├── schemas/
│   │   ├── services/
│   │   ├── reconciliation/
│   │   ├── ai/
│   │   ├── guardrails/
│   │   ├── database/
│   │   └── main.py

```

```
|
|  └─ tests/
|
|  └─ data/
|      ├── development/
|      ├── evaluation/
|      └─ demo/
|
|  └─ scripts/
|      └─ generate_data.py
|
|  └─ docs/
|      ├── 01_PRODUCT_SPEC.md
|      ├── 02_SYSTEM_ARCHITECTURE.md
|      └─ 03_DATA_SPEC.md
|
|  └─ .env.example
|  └─ .gitignore
|  └─ README.md
```

22. Implementation Boundary

We will NOT initially build:

- Microservices
- Kubernetes
- Real Razorpay API integration
- Real bank integrations
- Real financial transactions
- Mobile applications
- Complex ML models
- Multi-agent architecture
- Multiple unrelated finance workflows
- Refund processing
- Tax reconciliation
- Cash forecasting

The project will remain focused on:

Multi-source settlement reconciliation + AI-powered exception investigation + bounded resolution + human review.

23. Initial Implementation Sequence

The actual development sequence will be:

1. Create GitHub Repository
- ↓
2. Scaffold Frontend
React + Vite + Tailwind
- ↓
3. Scaffold Backend
Python + FastAPI
- ↓
4. Connect Frontend ↔ Backend
- ↓
5. Configure SQLite + SQLAlchemy
- ↓
6. Create Database Models
- ↓
7. Build Synthetic Data Generator
- ↓
8. Build Data Validation
- ↓
9. Build Deterministic Reconciliation
- ↓
10. Build Exception Classification
- ↓
11. Build Evidence Builder
- ↓
12. Add AI Investigation
- ↓
13. Add Guardrails
- ↓
14. Add Human Review
- ↓
15. Add Audit Trail
- ↓
16. Build Dashboard
- ↓
17. Add Settlement Q&A
- ↓
18. Evaluate + Test
- ↓
19. Deploy
- ↓
20. Prepare Razorpay Submission

Most important rule:

The first working version will NOT contain AI.

We first need:

Synthetic data → validation → deterministic reconciliation → correct results.

Once that foundation works reliably, we add the AI investigation layer on top of it.

This keeps the project controllable and ensures that if the AI component has problems, **we still have a working finance reconciliation product underneath it.**